

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	COMPUTACIÓN GRAFICA, VISIÓN COMPUTACIONAL Y MULTIMEDIA (E)				
TÍTULO DE LA PRÁCTICA:	<i>Motores de Física y Colisiones</i>				
NÚMERO DE PRÁCTICA:	<i>01</i>	AÑO LECTIVO:	<i>5to</i>	NRO. SEMESTRE:	<i>NOVENO</i>
FECHA DE PRESENTACIÓN	<i>dd/mm/aaaa</i>	HORA DE PRESENTACIÓN	<i>hh/mm/ss</i>		
INTEGRANTE (s) <i>Sencia Ale Bryan Daniel</i>				NOTA (0-20)	
DOCENTE(s): MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.					

RESULTADOS Y PRUEBAS
I. DESGLOSE DEL PROYECTO <i>ESCAPECITY</i> DE KALE a. IMPLEMENTACIÓN: ENTIDAD PRINCIPAL (KALE) Cuando desarrollamos el personaje principal hemos aplicado transformaciones geométricas para gestionar el movimiento, la orientación y el cambio de estado biomórfico (humano-lobo). Aquí es donde le damos vida al personaje usando pura matemática afín: $P' = AP + W$ i. Traslación (W): Movimiento y Salto La traslación es básicamente mover un punto a otro sumándole un vector W. En el código, no teletransportamos al personaje, sino que calculamos cuánta "fuerza" de movimiento (W) le aplicamos en cada frame. Análisis del Código: Para el movimiento horizontal, multiplicamos el input del jugador por una constante de velocidad, mientras que para el salto aplicamos un impulso vertical instantáneo. <pre>// Traslación Horizontal (FixedUpdate) Rigidbody2D.linearVelocity = new Vector2(Horizontal * Speed, Rigidbody2D.linearVelocity.y); // Traslación Vertical (Salto) Rigidbody2D.AddForce(Vector2.up * JumpForce, ForceMode2D.Impulse);</pre> Sustento Matemático: Cada frame, la nueva posición P' se obtiene sumando el desplazamiento acumulado al vector de posición original P: $P' = P + (V \cdot \Delta t)$

Donde V es el vector de velocidad resultante y Δt es el *Time.deltaTime*



Img. 01 - Línea roja del Debug.DrawRay

ii. Reflexión y Escalado (A): El "Giro" del Personaje

Para que Kale pueda mirar a la izquierda, no usamos rotación, sino una reflexión horizontal. Esto se logra mediante un escalado negativo en el eje X de la matriz A .

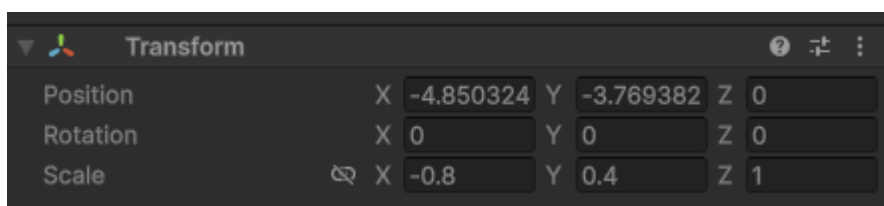
Análisis del Código:

Guardamos la escala original en *Start* y la invertimos multiplicando por -1 cuando el input es negativo.

```

if (Horizontal > 0) {
    transform.localScale = originalScale; // Escala original (0.8, 0.4, 1)
}
else if (Horizontal < 0) {
    // Inversión del eje X para aplicar reflexión
    transform.localScale = new Vector3(-originalScale.x,
    originalScale.y, originalScale.z);

```



Img. 02 - Reflexión del personaje y valores del inspector en ese momento.

Sustento Matemático:

Se aplica una transformación lineal donde la matriz A tiene un determinante negativo, invirtiendo el sentido de los vectores en el eje de las abscisas:

$$A = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}$$

iii. Transformación de Estado (Metamorfosis Humano-Lobo)

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

Esta mecánica es una composición de transformaciones que altera tanto la visibilidad como la geometría de colisión para adaptarse a la nueva forma.

- Traslación del Sensor (Raycast): El centro de masa cambia, por lo que el vector de traslación del rayo de suelo debe acortarse de 2.1 unidades a 0.8 unidades para que la detección sea precisa en modo lobo.
- Escalado de Colliders: Se desactivan y activan componentes para cambiar la "matriz de colisión" de una cápsula (humano) a una caja (lobo).

```
void ActualizarEstadoForma() {
    visualHumano.SetActive(!esLobo); // Escalado lógico (visibilidad)
    visualLobo.SetActive(esLobo);

    colliderHumano.enabled = !esLobo; // Cambio de geometría de colisión
    colliderLobo.enabled = esLobo;

    // Traslación dinámica del sensor de suelo
    RayDistance = esLobo ? rayDistLobo : rayDistHumano;
}
```



Img. 03 - Diferencia de colliders de ambos estados del personaje

iv. Composición de Rotación y Traslación: El Ataque (Onda)

En esta mecánica se demuestra cómo una rotación inicial altera el sentido de una traslación constante posterior. El proyectil no "sabe" si va a la izquierda o derecha; simplemente se traslada hacia su eje "derecho" local, el cual nosotros orientamos previamente.

Sustento Matemático.

Para que la onda viaje en sentido opuesto al eje Y global, aplicamos una matriz de rotación A sobre el eje vertical Y .

- Rotación (A): Aplicamos $R_y(180^\circ)$. Esto invierte la dirección del vector unitario horizontal del objeto.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

- Composición: La posición final P' es el resultado de rotar el objeto y luego sumarle el vector de traslación W en cada frame:

$$P' = (R_y.P) + W$$

Análisis del Código.

En el script KaleMovement.cs en la función *Attack()*, primero calculas la posición de aparición (spawn) y luego decides si el proyectil debe rotarse basándote en la escala de Kale:

```
void Attack() {
    //Traslación del punto de origen (W) para que no salga del centro
    de Kale
    float offsetY = esLobo ? 0.1f : 0.3f;
    Vector3 spawnPos = transform.position + new Vector3(direccion *
    0.1f, offsetY, 0);
    //Instanciación con rotación de identidad (0,0,0)
    GameObject ondaInstancia = Instantiate(OndaPrefab, spawnPos,
    Quaternion.identity);
    //Rotación Afín: Si miramos a la izquierda, rotamos 180 grados en Y
    if (direccion < 0) {
        ondaInstancia.transform.rotation = Quaternion.Euler(0, 180, 0);
    }
}
```



Img. 04 - Rotación de la onda de ataque dependiendo a donde se dirige Kale

b. XD

CONCLUSIONES

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

METODOLOGÍA DE TRABAJO

Colocar la metodología de trabajo que ha utilizado el estudiante o el grupo para resolver la práctica, es decir el procedimiento/secuencia de pasos en forma general.

REFERENCIAS Y BIBLIOGRAFÍA

Colocare las referencias utilizadas para el desarrollo de la práctica en formato IEEE o APA